

2026 Digital IC Design

Homework 4: Synchronous FIFO & Four-Banked FIFO Interface

1. Introduction

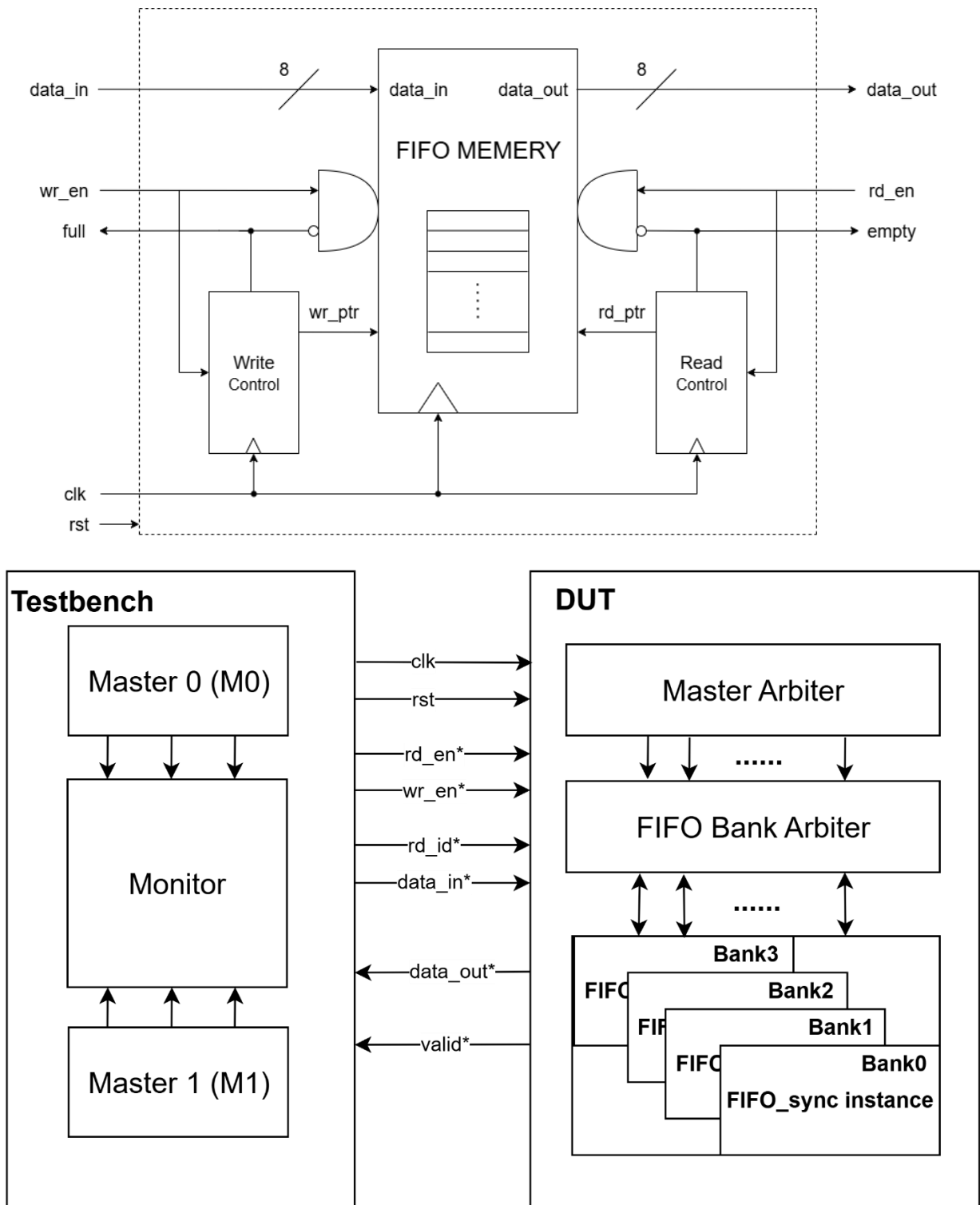
Synchronous FIFO (First-In, First-Out) is a type of buffer memory used in digital design to temporarily store data between two processes that operate under the same clock domain. Its primary function is to manage data flow and rate matching between a writer and a reader process. Data is written to one end and read from the other, maintaining the order in which it was received (FIFO principle). Key status flags, such as Full and Empty, prevent data overflow and underflow.

The **Four-Bank FIFO Interface** is a composite buffer architecture consisting of **four independent 8×32 synchronous FIFO memory**, coordinated under a shared arbitration and access control mechanism. It enables two masters to perform data transactions in an organized and conflict-free manner, while maintaining predictable access behavior and data consistency across all banks.

The Four-Bank FIFO serves as a shared data caching subsystem for **two independent masters (Master0 and Master1)**. Each master can request to perform either **read** or **write** operations in any clock cycle. The system arbitrates access between the two masters and routes the transactions to the appropriate FIFO bank according to predefined policies.

2. Specification

2.1. System Block Diagram for Synchronous FIFO & Four-Bank FIFO Interface



2.2. System I/O Interface

Signal Name	I/O	Width	Description
Synchronous FIFO			
<i>clk</i>	Input	1	System clock signal. All operations are triggered on the positive edge of this clock.
<i>rst</i>	Input	1	System reset signal. Active-high, asynchronous reset.
<i>wr_en</i>	Input	1	Write Enable. When this signal is high AND the FIFO is not full (full is 0), the data present on <i>data_in</i> is written into the FIFO memory.
<i>rd_en</i>	Input	1	Read Enable. When this signal is high AND the FIFO is not empty (empty is 0), the oldest data stored in the FIFO is output onto <i>data_out</i> .
<i>data_in</i>	Input	8	8-bit unsigned input data. This is the data being supplied to the FIFO to be written. This data is only loaded into the memory when <i>wr_en</i> is active.
<i>full</i>	Output	1	Full Status Flag. This signal is high when the FIFO's memory is completely filled to its maximum capacity.
<i>empty</i>	Output	1	Empty Status Flag. This signal is high when the FIFO contains no data words.
<i>data_out</i>	Output	8	8-bit unsigned output data. This is the data retrieved from the FIFO. It presents the oldest word that was written.
Four-Bank FIFO Interface			
<i>clk</i>	Input	1	System clock signal. All operations are triggered on the positive edge of this clock.
<i>rst</i>	Input	1	System reset signal. Active-high, asynchronous reset.
<i>wr_en_M0</i> <i>wr_en_M1</i>	Input	1	Write request from masters. When this signal is high AND the FIFO is not full (full is 0), the data present on <i>data_in*</i> is written into the Four-Bank FIFO memory.
<i>rd_en_M0</i> <i>rd_en_M1</i>	Input	1	Read request from masters. When this signal is high AND the FIFO is not empty (empty is 0), the oldest data stored in the destined bank of Four-Bank FIFO is output onto <i>data_out*</i> .
<i>data_in_M0</i> <i>data_in_M1</i>	Input	8	8-bit unsigned input data from masters. This is the data being supplied to the FIFO to be written. This data is only loaded into the memory when <i>wr_en*</i> is active AND the destined bank FIFO <i>full</i> signal is low.
<i>rd_id_M0</i> <i>rd_id_M1</i>	Input	2	Read bank ID from masters. Read operation should based on this ID to fetch the destined bank of FIFO memory.
<i>data_out_M0</i> <i>data_out_M1</i>	Output	8	8-bit unsigned output data to masters. This is the data retrieved from the Four-Bank FIFO. It presents the oldest word that was written based on <i>rd_id*</i> .
<i>valid_M0</i> <i>valid_M1</i>	Output	1	Valid signal to masters. This signal should be high when the output data from <i>data_out*</i> is valid.

2.3. Synchronous FIFO Specification

- Data Width: 8 bits
- Depth: 32 entries
- Access Order: In-order

- Clock Domain: Single clock (synchronous for both read and write)
- Concurrent Operations: Supports simultaneous read and write in the same clock cycle.
- Read/Write Latency:
 - Write: Data is stored into memory with one-cycle latency.
 - Read: Data is available at the output with one-cycle latency after read enable is asserted.
- Protection Mechanisms:
 - Overflow Protection: Write operations are blocked when FIFO is full.
 - Underflow Protection: Read operations are blocked when FIFO is empty.
- Data Integrity: Ensures no data corruption or loss under any legal operation sequence; every valid write is eventually read exactly once in the same order.

2.4. Four-Bank FIFO Interface specification

- Number of Banks: 4 independent FIFO banks
- Each Bank Specification (Should use FIFO_sync module):
 - A. Block Size: 8 bits
 - B. Depth: 32 entries
 - C. Memory Space: 128B
 - D. Synchronous Operation: Single clock domain shared among all banks.
 - E. Supports concurrent read/write within the same cycle.
 - F. In-order access, with one-cycle latency for both read and write.
- Master Arbitration:
 - Number of Masters: 2 (Master0, Master1)
 - Arbitration Policy: Round-Robin (RR)
 - A. When both masters issue requests in the same cycle, access is granted alternately to ensure fairness.
 - B. If only one master requests, it is granted immediate access without arbitration.
 - C. At any given cycle, only one master can perform a transaction (read or write).
- Write Policy (LRU-Based Bank Selection):
 - Bank Selection:
 - A. Write operations are directed to the Least Recently Used (LRU) bank.
 - LRU Update Rule:
 - A. After each successful write, the written bank becomes the most recently used, and the LRU order is updated accordingly.

B. If the selected LRU bank is full, the write operation is not executed, and the LRU order remains unchanged (no update).

➤ Write Protection:

A. Overflow is prevented by checking the bank's full flag before write.

- Read Policy (Bank-ID Directed)

- Each master specifies the target bank ID when issuing a read request.
- The system routes the read request to the corresponding bank.
- Read operations do not affect the LRU ordering.
- If the selected bank is empty, the read operation is blocked and no data is returned.
- Read latency is one clock cycle.

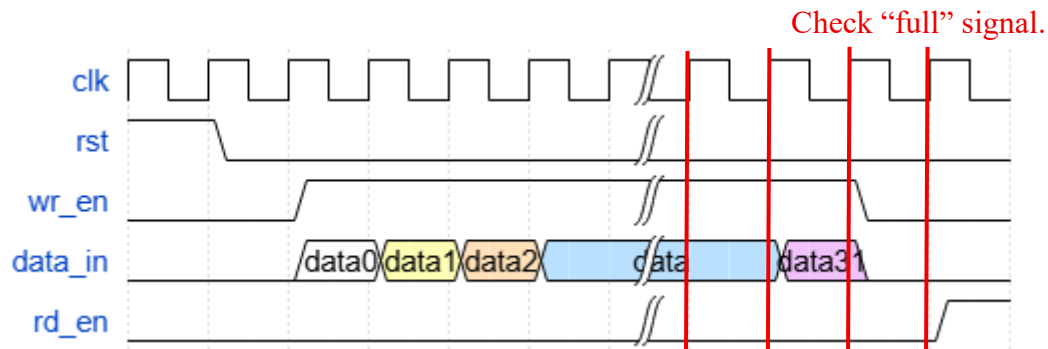
- Data Integrity and Protection

- No overflow or underflow occurs under valid operation.
- Data ordering is strictly preserved within each bank (FIFO behavior).
- Each valid write is guaranteed to be readable once and only once.
- Arbitration ensures that no two masters access the same resource simultaneously.

2.5. Input and Output Timing Specification

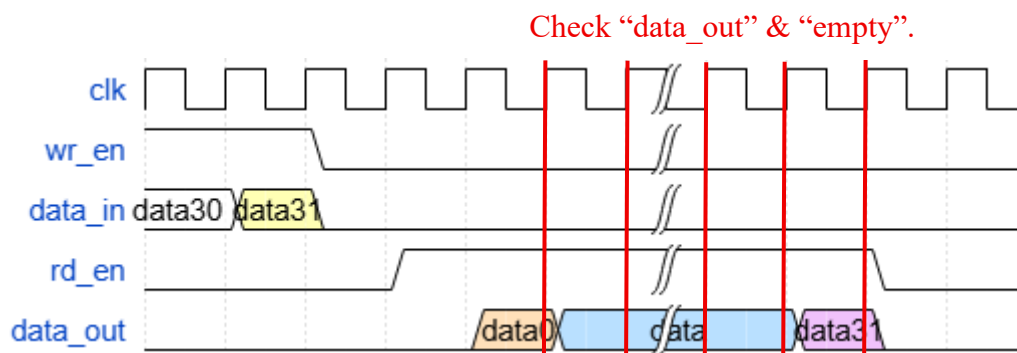
When the host asserts the **wr_en** signal, the host begins transmitting input data on the **data_in** bus at each clock cycle, as illustrated in the timing diagram below. The **data_in** signal from the host is an unsigned 8-bit value.

Whenever the write transaction is processed, the **full** signal is verified at the subsequent **rising edge** of the clock.

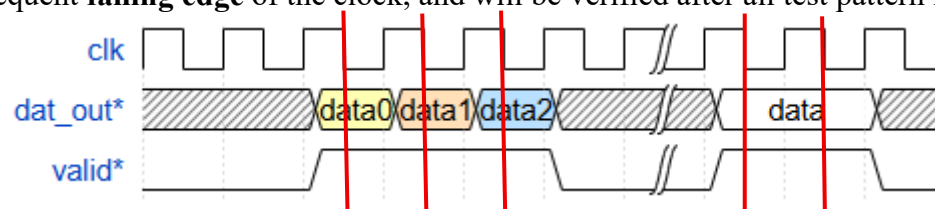


After the data undergoes FIFO, the **rd_en** signal being asserted high indicates that the **data_out** bus is actively transmitting data to the host. In this testbench, output data verification begins at the second **rising edge** after **rd_en** is asserted, and continues at each subsequent **rising edge**. The timing specification for output verification is shown in the figure below.

Whenever the read transaction is processed, the **empty** signal is verified at the subsequent **rising edge** of the clock.



While the data is read from Four-Bank FIFO Interface, it will be record into the monitor for the comparison with golden data. Monitor will record the data fetched by master and the master index of this access. Monitor record data according to the **valid** signal, the data will be sent into monitor at the subsequent **falling edge** of the clock, and will be verified after all test pattern is completed.



2.6 File Description

File Name	Description
FIFO_sync.v	The top module of Synchronous FIFO.
FourBankFIFO.v	The top module of Four-Bank FIFO Interface.
testfixture.sv	Testbench file for FIFO_sync.v.
testfixture_bank.sv	Testbench file for FourBankFIFO.v.
dat/fifo_sim.hex	Test data for FourBankFIFO.v.
dat/golden.hex	Golden data for FourBankFIFO.v.

3. Scoring:

3.1. Functional Simulation of Synchronous FIFO [40%]

In this assignment, the functional simulation is divided into three test cases. A full score of 60 points will be awarded only if all three cases pass successfully. If any single case fails, the score for this part will be zero. After completing the simulation, the results will be displayed in the ModelSim terminal, as illustrated in the figure below.

Please don't design specifically for the test pattern. Otherwise, you will get 0 point.

```
# =====
#      ( \_./ )
#      ( o I o )      Congratulations !!
# o-(      )-o      Simulation PASS !!
#      ' '
# =====
```

3.2. Functional Simulation of Four-Bank FIFO Interface [30%]

In this assignment, the functional simulation is divided into three test cases. A full score of 60 points will be awarded only if all three cases pass successfully. If any single case fails, the score for this part will be zero. After completing the simulation, the results will be displayed in the ModelSim terminal, as illustrated in the figure below.

Please don't design specifically for the test pattern. Otherwise, you will get 0 point.

```
# =====
#      ( \_./ )
#      ( o I o )      Congratulations !!
# o-(      )-o      Simulation PASS !!
#      ' '
# =====
```

3.3. Pre-Layout Simulation of Synchronous FIFO [30%]

3.3.1. Synthesis

Your code should be synthesizable. After it is synthesized in Quartus, files named FIFO_sync.vo and FIFO_sync_v.sdo will be obtained.

DEVICE : Cyclone IV E - EP4CE55F23A7

3.3.2. Simulation

In the pre-layout simulation, all three test cases must also pass successfully to obtain the remaining 30 points. All results should be generated correctly using FIFO_sync.vo and FIFO_sync_v.sdo. Upon successful simulation, the corresponding message will appear in the ModelSim terminal.

```
# =====
#      ( \_./ )
#      ( o I o )      Congratulations !!
#  o-(      )-o      Simulation PASS !!
#      ' ' '
# =====
```

3.4. Bonus: Pre-Layout Simulation of Four-Bank FIFO Interface [10%]

3.4.1. Synthesis

Your code should be synthesizable. After it is synthesized in Quartus, files named FourBankFIFO.vo and FourBankFIFO_v.sdo will be obtained.

DEVICE : Cyclone IV E - EP4CE55F23A7

3.4.2. Simulation

In the pre-layout simulation, all three test cases must also pass successfully to obtain the remaining 30 points. All results should be generated correctly using FourBankFIFO.vo and FourBankFIFO_v.sdo. Upon successful simulation, the corresponding message will appear in the ModelSim terminal.

```

# =====
#      ( \_./ )
#      ( o I o )      Congratulations !!
#      o-(      )-o      Simulation PASS !!
#      ' ' ' '
# =====

```

4. Submission

4.1. File Submission

You should classify your files into two directories and compress them to .zip format. The naming rule is StudentID.zip. **If your file is not named according to the naming rule, you will lose five points.**

	RTL
*.v	All of your Verilog RTL code
	Pre-Layout
*.vo	Gate-Level netlist generated by Quartus
*.sdo	SDF timing information generated by Quartus
	Documentary
Report.pdf	The report file of your design (in pdf)

```

📁 StudentID.zip
  📁 StudentID
    📁 RTL
    📄 *.v (All of your Verilog RTL code)
    📁 Pre-Layout
    📄 *.vo, *.sdo
    📁 Documentary
    📄 Report.pdf

```

4.2. Report File

Please follow the spec of report. You are asked to describe how the circuit is designed as detailed as possible, and the flow summary result is necessary in the report. Please fill the field of total logic elements, total register, total memory bits, embedded multiplier 9-bit elements according to the flow summary of your synthesized design. And fill the field of clock cycle used and clock width according

to the pre-layout simulation results that Modelsim shows if the design passes pre-layout simulation. **If the filled-in values don't match your synthesized design or pre-layout simulation results, you will lose 5 points.**

Flow Summary	
<<Filter>>	
Flow Status	Successful - Tue Oct 07 14:53:45 2025
Quartus Prime Version	20.1.1 Build 720 11/11/2020 SJ Lite Edition
Revision Name	FIFO_sync
Top-level Entity Name	FIFO_sync
Family	Cyclone IV E
Device	EP4CE55F23A7
Timing Models	Final
Total logic elements	361 / 55,856 (< 1 %)
Total registers	276
Total pins	22 / 325 (7 %)
Total virtual pins	0
Total memory bits	0 / 2,396,160 (0 %)
Embedded Multiplier 9-bit elements	0 / 308 (0 %)
Total PLLs	0 / 4 (0 %)

4.3. Notes

- In this homework, **you are not allowed to modify the defined “CYCLE” in testbench file.** Please do not modify any content of the testbench.
- Please submit your .zip file to folder HW4 in moodle.
Deadline: 2025/12/07 23:55
- Late submission will only be accepted within a week and will result in a penalty of 5 points per day. Late submission of bonus part will not be accepted.
- If you complete the bonus part, you should fill in your name and student ID on an Excel provided on Moodle, TA will only check the code according to the name on the Excel.
- TA email:
 - m56134028@gs.ncku.edu.tw (for Synchronous FIFO)
 - m18131507@gs.ncku.edu.tw (for Four-Bank FIFO Interface)